

Prof. Dr.-Ing. M. Dahm

OOP2

**Termin 02 abstract, interface**

*In diesem Praktikum üben Sie den Umgang mit abstrakten Klassen und Interfaces, wie Sie sie zum Ende des letzten Semesters in OOP1 kennen gelernt haben. Sie sollten also entweder noch wissen, worum es geht, oder Sie schlagen das Thema noch einmal nach.*

Für diesen Termin bekommen Sie das **Testat** für Ihre Anwesenheit im Termin, sowie für den Nachweis, dass Sie die Aufgaben bearbeitet haben.

Den Nachweis erbringen Sie durch Zeigen der Aufgaben in der Gruppe. Sie teilen dafür Ihren Desktop oder nur das VSCode-Fenster. So können die Betreuer\*innen sehen, was Sie bearbeitet haben.

Wenn sie **Fragen zu den Aufgaben oder der Lösung** haben, fragen Sie!

**Tun Sie's!** Wer viel fragt, bekommt viel Antwort.

Sie profitieren selber auch von den Fragen und Antworten der anderen.

**Tipps:**

Sie können Ihre Lösung mit wenig Aufwand dadurch **kommentieren**, indem Sie die passenden Texte aus der Aufgabe in Ihren Code kopieren und ggf. mit eigenen Anmerkungen erweitern.

```
/** Verwenden Sie das javadoc-Format für Klassen und Methoden
 * (incl. @param und @return) */
und
// Zeilenkommentare für einzelne Schritte.
```

Wenn Sie Ihre Lösung so vorbereiten, können Sie daraus schnell den passenden Code entwickeln.

Auch, wenn VSCode nicht mehr die **Ausgabe der passenden Überschrift auf die Konsole** generiert, sollten Sie für das einfachere Testen diese selber als `S.o.p()`-Anweisung einfügen.

-----

Definieren Sie für jeden Praktikumstermin **ein eigenes Java-Projekt** (Siehe Anleitung zu VSC)

-----

Bearbeiten Sie IMMER die Aufgaben in der angegebenen **Reihenfolge**, sie *bauen aufeinander auf!*

-----

Wenn in den Aufgaben **Tests** gefordert sind, müssen Sie die natürlich **durchführen**.

UND: Wenn sich bei den Tests ergibt, dass die getesteten Klassen bzw. Methoden nicht das tun, was sie sollen, dann müssen Sie diese Methoden **solange korrigieren, bis die Anforderungen erfüllt sind!**

*Zunächst üben Sie den Umgang mit einer Abstrakten Klasse, und die Auswirkungen des abstrakt-Seins auf die Vererbung und die Instanziierung.*

## 1 Abstrakte Klasse `Grafik`.

- a) **Definieren** Sie eine Klasse `Grafik` (Siehe Anleitung zu VSC) mit Eigenschaften einer Grafik, zunächst mit Daten, z.B. für die Position `x` und `y` (als Zahlenwert) sowie die `farbe` (String) einer `Grafik`

Diese Eigenschaften sollen außerhalb der Klasse nicht sichtbar sein, auch nicht in den Subklassen (!).

- b) **Definieren** Sie als weitere Eigenschaften eine Methode, die eine `Grafik` in `x` und `y` Koordinaten positioniert, sowie eine Methode, die die Farbe der `Grafik` setzt.

- c) **Definieren** Sie eine Methode, die Eigenschaften (Position und Farbe) einer `Grafik` zu Testzwecken als String zurückgibt.

*Hinweis:* Hierzu redefinieren Sie die Standardmethode `public String toString()`.

- d) **Definieren** Sie einen Konstruktor, mit dem man alle diese Eigenschaften setzen kann.

*Hinweis:* nutzen Sie dafür die oben definierten Methoden

- e) **Erstellen** Sie eine weitere Klasse: `Test`, um die oben definierten Methoden zu testen. In deren `main()`-Methode definieren Sie nun ein `Grafik`-Objekt, verschieben es, ändern die Farbe und geben danach jeweils die Eigenschaften des `Grafik`-Objekts auf die Konsole aus.

**Führen** Sie nun zur Kontrolle und **zum Testen** die Klasse `Test` aus (Run->Run)

- f) **Definieren** Sie nun in der Klasse `Grafik` eine abstrakte Methode `draw()`, um etwas zu zeichnen.

-> Welchen Hinweis der IDE VSCode bekommen Sie nach der Definition dieser abstrakten Methode?

-> Diese Hinweise, Meldungen und Warnungen stehen links als Icon

-> gehen Sie mit der Maus über das Icon um mehr zu erfahren

**Frage:** Warum gibt es zu Recht diese Fehlermeldung? Was müssen Sie tun, um diese Meldung zum Verschwinden zu bringen (Die abstrakte Methode soll natürlich abstrakt bleiben!) ?

## 2 Konkrete Subklasse `Quadrat`

Definieren Sie die Klasse `Quadrat` als Subklasse von `Grafik`.

- a) Bereits beim Definieren von `Quadrat` als Subklasse meldet die IDE VSCode wieder einen Fehler durch ein Icon am linken Rand:

**Beheben** Sie die angegebene Fehlermeldung sinnvoll, d.h. durch Definition der fehlenden Methode, die hier in der Implementierung nur etwas auf die Konsole ausgeben braucht.

- b) **Definieren** Sie die Länge des `Quadrats` als zusätzliche, natürlich private, Eigenschaft, sowie eine Methode, um die Länge zu ändern.

- c) **Definieren** Sie einen Konstruktor, der Position, Länge und Farbe des `Quadrats` setzt.

- d) **Fügen** Sie in `main()` der Klasse `Test` einige Aktionen hinzu: definieren Sie ein `Quadrat`, verschieben Sie es, und ändern Sie die Länge mit den passenden Methoden. Geben Sie jeweils die Eigenschaften des Quadrats mit einer passenden Methode auf die Konsole aus. (Ergänzen Sie ggf. diese Methoden in `Quadrat`. Ändern Sie dafür nicht (!) die Sichtbarkeiten der privaten IV in `Grafik`).
- e) **Testen** Sie in `main()` der Klasse `Test` auch den Aufruf der Methode `draw()` für das `Quadrat`.

*Nun erstellen Sie zwei weitere Klassen. Beide implementieren das Interface `Rollbar`, aber nur eine Klasse erbt von der Klasse `Grafik`. Diese Klassen verdeutlichen bei der Anwendung, dass ihr Datentyp sowohl durch ihre Vererbung als auch durch ihre implementierten Interfaces bestimmt wird.*

### 3 Interface Rollbar

- a) **Definieren** Sie ein Interface `Rollbar` mit der Methode `public int rollen();`
- b) **Versuchen** Sie, in diesem Interface eine Variable `double geschwindigkeit` zu definieren.  
Diese soll angeben wie schnell das Objekt gerollt werden soll.
- Welche Fehlermeldung erhalten Sie nun von der IDE?
  - Entfernen Sie diese Variable wieder.
- c) **Frage:** Wie könnte man trotzdem im Interface festlegen, dass man die Geschwindigkeit von außen einstellen kann, mit der gerollt wird (es gibt mehrere Möglichkeiten).

### 4 Implementierende Klassen Kreis und Tonne

- a) **Definieren** Sie eine Klasse `Tonne`, die das Interface `Rollbar` implementiert.
- b) **Frage:** Welche Methode müssen (!) Sie in der Klasse `Tonne` implementieren, damit keine Fehlermeldung erscheint? Warum?
- c) **Definieren** Sie eine Klasse `Kreis` als Subklasse von `Grafik`, mit einem `Radius` als Instanz-Variable. Außerdem soll die Klasse die beiden Interfaces `Rollbar` und `java.lang.Comparable` (aus der Java-API, siehe Anmerkungen unten <sup>1</sup>) implementieren.

Sie verwenden hier `java.lang.Comparable`, um zu üben, ein bereits vorhandenes Interface zu implementieren. Außerdem implementiert auch die Klasse `String` dieses Interface, was Sie in der nächsten Aufgabe ausnutzen.

---

<sup>1</sup> Da Sie noch nicht wissen, wie Sie mit generischen Datentypen umgehen, geben Sie **keinen Typ-Parameter** an. Sie verwenden das Interface so, wie sie das von anderen Interfaces auch kennen, nur mit seinem Namen. Ignorieren Sie auch bis auf weiteres **Warnungen** von VSCode bezüglich „**Rawtype**“ und „**Generics**“. Was es damit auf sich hat, behandeln wir später in diesem Semester, ebenso wie das Interface `java.lang.Comparable` ausführlich behandelt werden wird.

Die mit `Comparable` notwendige Methode **implementieren** Sie so, dass sie immer 0 zurückgibt.

Sie brauchen und sollten sich jetzt noch keine echten Berechnungen ausdenken. Um z.B. auf den Radius zuzugreifen, müssen Sie das Konzept Generics anwenden, was wir später im Semester behandeln.

d) **Zeichnen** Sie ein UML-Klassendiagramm aller bisher verwendeten Klassen und Interfaces. Dafür können Sie Papier oder ein Zeichentool (z.B. MS Paint, Word, draw.io) verwenden.

Auch diese Darstellung verdeutlicht, dass der Datentyp einer Klasse sowohl durch ihre *Vererbung* als auch durch ihre implementierten *Interfaces* bestimmt wird.

## 5 Polymorphismus mit Interfaces

*In den folgenden Aufgaben machen Sie sich klar, dass Sie mit Hilfe von Interfaces **typischer** Methoden für Objekte von Klassen aufrufen können, die – außer `Object` – keine spezielle gemeinsame Superklasse (und auch sonst nicht viel gemeinsam) haben. Andernfalls wären so unterschiedliche Objekte wie `Strings` oder `Kreise` nicht definiert untereinander zu vergleichen. Das sind der Sinn und die Stärke von Interfaces.*

a) **Definieren** Sie in `main()` in der Klasse `Test` eine Variable `meineRolle` vom Typ `Rollbar`. Lassen Sie `meineRolle` zuerst auf einen `Kreis` zeigen und rufen Sie die Methode `rollen()` auf. Dann lassen Sie `meineRolle` auf eine `Tonne` zeigen und rufen Sie wiederum `rollen()` auf. Geben Sie das Ergebnis von `rollen()` jeweils auf der Konsole aus.

b) **Definieren** Sie Sie in `main()` in der der Klasse `Test` die Variablen `comp1` und `comp2` vom Typ `Comparable`. Da die Klasse `String` das Interface `Comparable` implementiert, lassen Sie nun diese beiden Variablen zwei unterschiedliche `Strings` referenzieren<sup>2</sup>. Vergleichen Sie die beiden Variablen nun mittels der Methode `compareTo()` und geben Sie das Ergebnis aus.

c) **Lassen** Sie die beiden Variablen nun zwei Objekte der Klasse `Kreis` referenzieren, vergleichen Sie die beiden Variablen dann mittels der Methode `compareTo()` und geben Sie das Ergebnis aus.

d) **Lassen** Sie die beiden Variablen nun zwei Objekte der Klasse `Tonne` referenzieren. Warum klappt das hier nicht mehr?

Objekte der Klasse `Tonne` können zwar `rollen()` wie auch ein `Kreis`. Man kann sie aber nicht miteinander vergleichen, was mit `Kreisen` aber klappt. Diese Klassen verdeutlichen bei der Anwendung, dass ihr Datentyp sowohl durch ihre *Vererbung* als auch durch ihre implementierten *Interfaces* bestimmt wird.

Viel Spaß und Erfolg

---

<sup>2</sup> Wie oben schon gesagt, implementiert die Klasse `String` das Interface `java.lang.Comparable`. Dass das so ist, können Sie auch in der API-Dokumentation der Klasse `String` nachsehen.